

Image Classification

Image Classification

- 1. Model Introduction
- 2. Image Classification: Images
 - Preview
- 3. Image Classification: Video
 - Preview
- 4. Image Classification: Real-time Detection
 - 4.1 USB Camera
 - Preview
- Reference

Demonstrating **Ultralytics** image classification on images, videos, and real-time detection using Python.

1. Model Introduction

Image classification is the simplest of the three tasks, involving classifying an entire image into one of a set of predefined categories.

The output of an image classifier is a single class label and a confidence score. Image classification is very useful when you only need to know which category an image belongs to, without knowing the location or exact shape of objects within that class.

2. Image Classification: Images

Use yolo26n-cls_ncnn_model to predict images in the ultralytics26 folder (non-Ultralytics built-in images).

Enter the code folder:

```
cd /home/sunrise/ultralytics26/yahboom_demo
```

Run the code:

```
python3 04.classification_image.py
```

Preview

Output image location for YOLO recognition: /home/sunrise/ultralytics26/output/



Example code:

```
from ultralytics import YOLO

# Load a model
#model = YOLO("/home/sunrise/ultralytics26/yolo26n-cls.pt")
model = YOLO("/home/sunrise/ultralytics26/yolo26n-cls_ncnn_model")

# Run batched inference on a list of images
results = model("/home/sunrise/ultralytics26/assets/dog.jpg") # return a list of Results
objects

# Process results list
for result in results:
    # boxes = result.boxes # Boxes object for bounding box outputs
    # masks = result.masks # Masks object for segmentation masks outputs
    # keypoints = result.keypoints # Keypoints object for pose outputs
    probs = result.probs # Probs object for classification outputs
    # obb = result.obb # Oriented boxes object for OBB outputs
    result.show() # display to screen
    result.save(filename="/home/sunrise/ultralytics26/output/dog_output.jpg") # save to
disk
```

3. Image Classification: Video

Use yolo26n-cls_ncnn_model to predict videos in the ultralytics26 folder (non-Ultralytics built-in videos).

Enter the code folder:

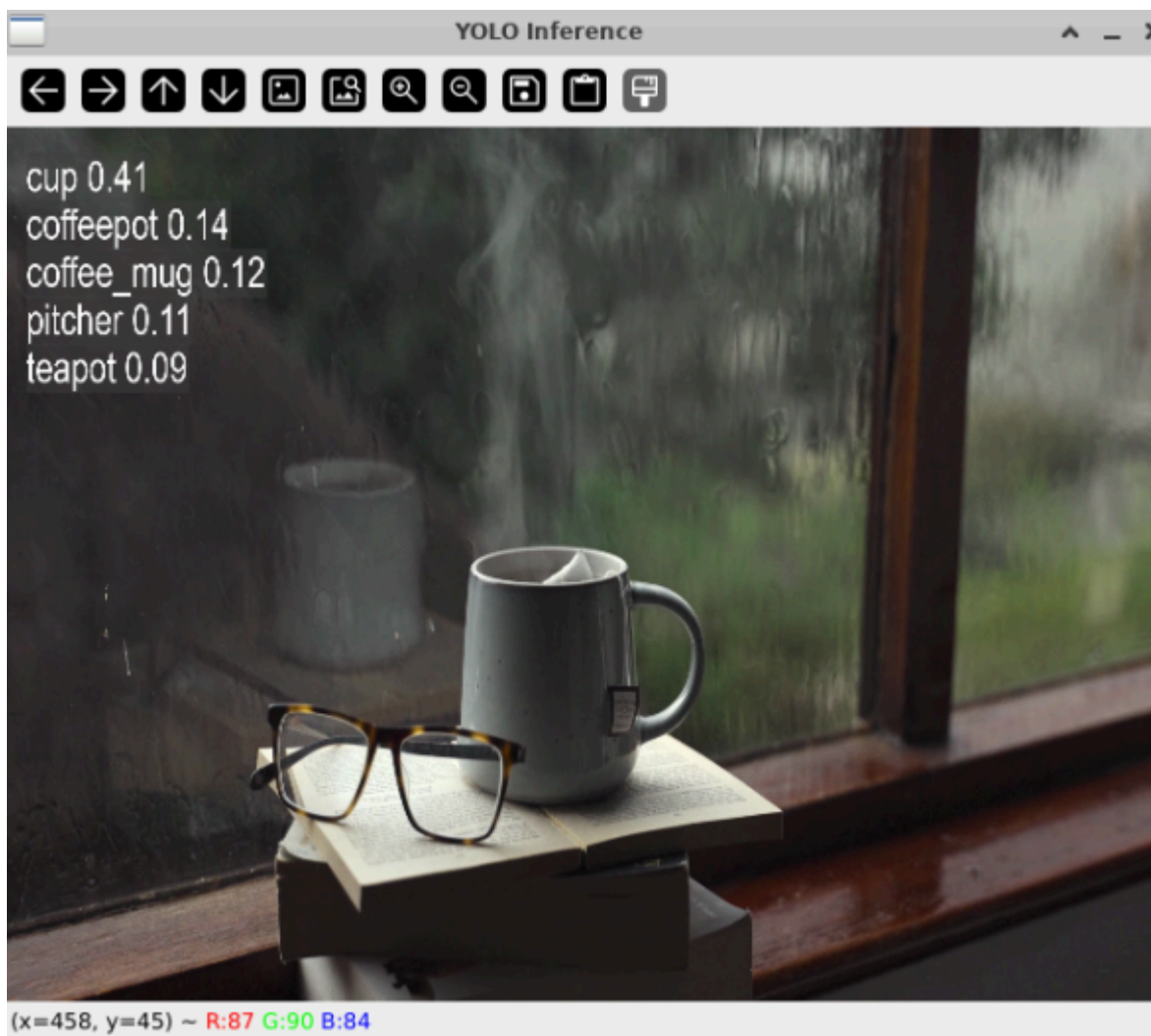
```
cd /home/sunrise/ultralytics26/yahboom_demo
```

Run the code:

```
python3 04.classification_video.py
```

Preview

Output video location for YOLO recognition: /home/sunrise/ultralytics26/output/



Example code:

```
import cv2
from ultralytics import YOLO

# Load the YOLO model
#model = YOLO("/home/sunrise/ultralytics26/yolo26n-cls.pt")
model = YOLO("/home/sunrise/ultralytics26/yolo26n-cls_ncnn_model")
```

```

# Open the video file
video_path = "/home/sunrise/ultralitics26/videos/cup.mp4"
cap = cv2.VideoCapture(video_path)

# Get the video frame size and frame rate
frame_width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
frame_height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fps = int(cap.get(cv2.CAP_PROP_FPS))

# Define the codec and create a Videowriter object to output the processed video
output_path = "/home/sunrise/ultralitics26/output/04.cup_output.mp4"
fourcc = cv2.VideoWriter_fourcc(*'mp4v') # You can use 'XVID' or 'mp4v' depending on your platform
out = cv2.VideoWriter(output_path, fourcc, fps, (frame_width, frame_height))

# Loop through the video frames
while cap.isOpened():
    # Read a frame from the video
    success, frame = cap.read()

    if success:
        # Run YOLO inference on the frame
        results = model(frame)

        # Visualize the results on the frame
        annotated_frame = results[0].plot()

        # Write the annotated frame to the output video file
        out.write(annotated_frame)

        # Display the annotated frame
        cv2.imshow("YOLO Inference", cv2.resize(annotated_frame, (640, 480)))

        # Break the loop if 'q' is pressed
        if cv2.waitKey(1) & 0xFF == ord("q"):
            break
    else:
        # Break the loop if the end of the video is reached
        break

# Release the video capture and writer objects, and close the display window
cap.release()
out.release()
cv2.destroyAllWindows()

```

4. Image Classification: Real-time Detection

4.1 USB Camera

Use yolo26n-cls_ncnn_model to predict USB camera feed.

Enter the code folder:

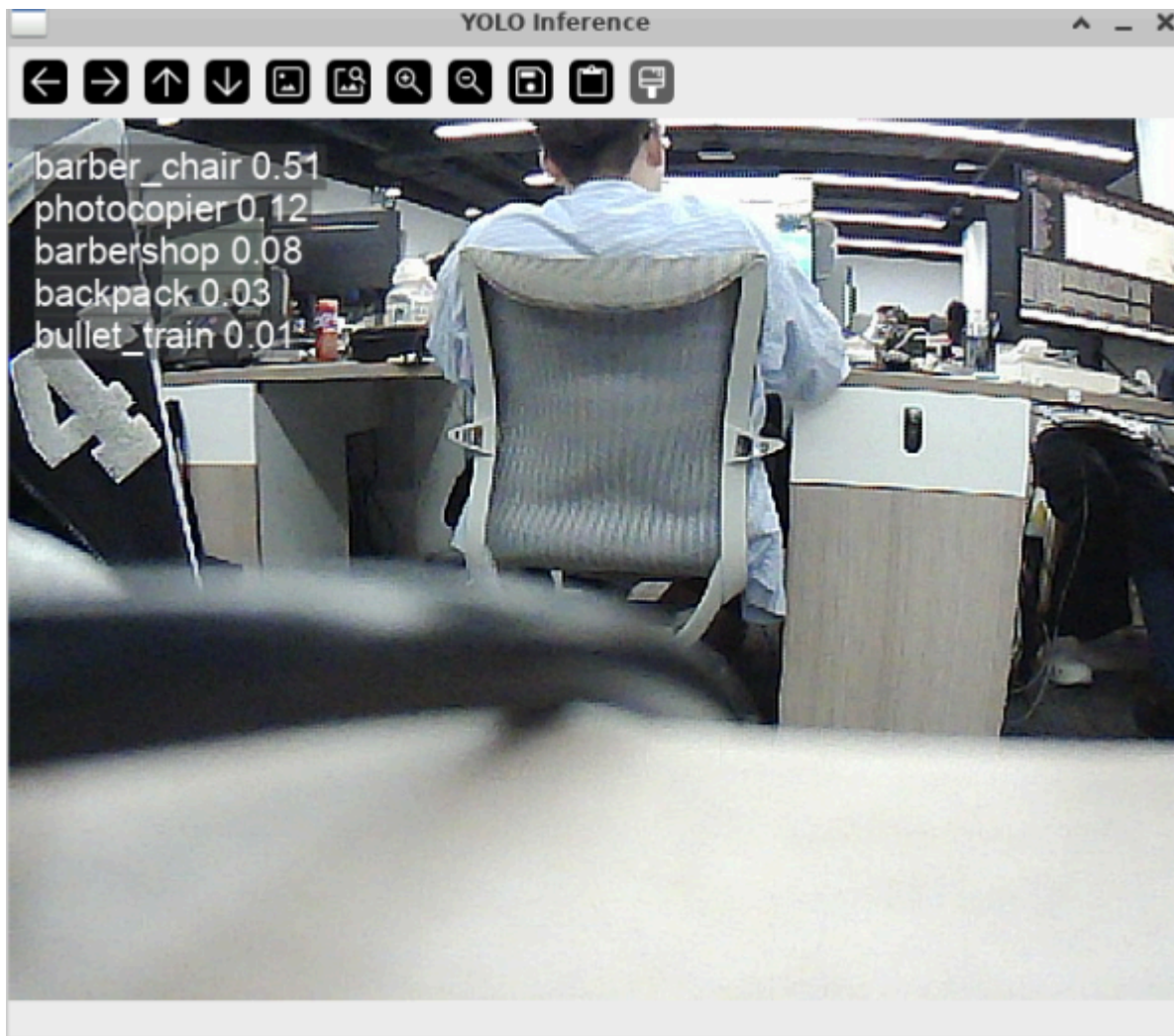
```
cd /home/sunrise/ultralytics26/yahboom_demo
```

Run the code: Click the preview window and press 'q' to terminate the program!

```
python3 04.classification_camera_usb.py
```

Preview

Output video location for YOLO recognition: /home/sunrise/ultralytics26/output/



Example code:

```
import cv2
from ultralytics import YOLO

# Load the YOLO model
```

```

#model = YOLO("/home/sunrise/ultralytics26/yolo26n-cls.pt")
model = YOLO("/home/sunrise/ultralytics26/yolo26n-cls_ncnn_model")

# Open the cammera
cap = cv2.VideoCapture(0)
cap.set(6, cv2.VideoWriter_fourcc('M', 'J', 'P', 'G'))
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)

# Get the video frame size and frame rate
frame_width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
frame_height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fps = int(cap.get(cv2.CAP_PROP_FPS))

# Define the codec and create a VideoWriter object to output the processed video
output_path = "/home/sunrise/ultralytics26/output/04.classification_camera_usb.mp4"
fourcc = cv2.VideoWriter_fourcc(*'mp4v') # You can use 'XVID' or 'mp4v' depending on your
platform
out = cv2.VideoWriter(output_path, fourcc, fps, (frame_width, frame_height))

# Loop through the video frames
while cap.isOpened():
    # Read a frame from the video
    success, frame = cap.read()

    if success:
        # Run YOLO inference on the frame
        results = model(frame)

        # Visualize the results on the frame
        annotated_frame = results[0].plot()

        # Write the annotated frame to the output video file
        out.write(annotated_frame)

        # Display the annotated frame
        cv2.imshow("YOLO Inference", cv2.resize(annotated_frame, (640, 480)))

        # Break the loop if 'q' is pressed
        if cv2.waitKey(1) & 0xFF == ord("q"):
            break
    else:
        # Break the loop if the end of the video is reached
        break

# Release the video capture and writer objects, and close the display window
cap.release()
out.release()
cv2.destroyAllWindows()

```

Reference

[Image Classification - Ultralytics YOLO Docs](#)

